OneMall: One Architecture, More Scenarios — End-to-End Generative Recommender Family at Kuaishou E-Commerce (2026)

arXiv:2601.21770v2 (cs.IR), 2 Feb 2026.

Challenges

- **Unifying multiple e-commerce scenarios in one recommender:** Kuaishou e-commerce spans *product-card*, *short-video*, and *live-streaming* item distributions, with different intent signals and item semantics; the paper targets a single end-to-end generative family across them.
- **Tokenizer requirements are unusually strict:** the "semantic token" must preserve real-world semantics *and* encode business-specific relations (e.g., product–product co-purchase and short-video–product watching-to-shopping links) across scenarios.
- **Serving-time constraints with long behavioral funnels:** exposure → click → buy histories are long; naïvely concatenating sequences makes Transformer inference expensive.
- **Bridging retrieval and ranking:** classic cascaded RecSys optimize a retrieval objective and a ranking objective separately; closing that gap requires a stable training pipeline to connect them.

Key Initiatives

- **E-commerce semantic tokenizer (multi-scenario):** learn item representations with an LLM-style multimodal backbone, then quantize to multi-level Semantic IDs (Res-Kmeans + FSQ) with scenario-specific embedding choices.
- **Pure Transformer-based decoder-only architecture:** use Query-Former compression for long user/item feature sequences, cross-attention for multi-behavior fusion, and Sparse MoE to scale total parameters while limiting activated parameters.
- **Retrieval–ranking connection via post-training RL:** use an online ranking model as a reward model and apply preference-optimization style RL (e.g., DPO/GRPO-family) on top of next-token (next-SID) training.
- **Industrial deployment at scale:** the system is reported as deployed and "serving over 400 million daily active users" and delivering GMV lifts in multiple surfaces.

Methodology

- **Semantic compression backbone (multimodal embedding):** the tokenizer first fine-tunes an LLM-style multimodal model using Item2Item alignment signals:
  - *Inputs:* product main image (224×224) + title; short-video samples 6 frames (224×224) + title/OCR/ASR.
  - *Backbone choices (as described):* Swin-Transformer as vision encoder and Qwen2.5-1.5B as the text encoder; the last hidden state of a special token `<EMB>` is used as the final item representation.
  - *Objective:* InfoNCE-style in-batch contrastive learning on mined positive pairs (product–product, short-video–product).
- **Semantic ID generation (quantization):** convert continuous embeddings to discrete Semantic IDs via a multi-stage pipeline (Figure 1).
  - *Res-Kmeans residual quantization:* apply residual K-means codebooks for the first quantization layers.
  - *FSQ last layer:* a 16-bit quantizer (4096 codes) on the residual is used to reduce code collisions and improve code distribution regularity.

  - *Scenario-specific embedding candidates:* product-card uses the product embedding; short-video concatenates product + short-video embeddings; live-streaming uses an item-tower embedding plus real-time selling product Semantic IDs as additional features.

- **Decoder-only generative recommender:** the model autoregressively predicts multi-level Semantic IDs, while reducing sequence costs with Query-Former compression and fusing multiple behaviors with cross-attention (Figure 2). Sparse MoE is used to scale parameters (the paper reports scaling up to 1.3B total parameters with 0.1B activated).

- **RL post-training for retrieval–ranking connection:** the paper treats the online ranking model as a reward model, and optimizes the generator with preference-based RL variants; in their RL setting they mention candidate generation with beam size 768 and balancing NTP and RL losses.

Experiments

- **Offline scaling (short-video scenario):** as total parameters scale from 0.05B to 1.3B-A0.1B, the paper reports Acc-SID1/2/3 improving from 14.5%/43.9%/61.0% to 16.2%/51.7%/71.7%, and simulation HR@50/100/500 improving from 32.9%/41.3%/60.5% to 45.6%/57.3%/76.0% (Table 1).

- **Simulation replay vs baselines:** OneMall (0.5B-A0.1B) outperforms SASRec and TIGER across product-card / short-video / live-streaming scenarios; e.g., Product-card HR@50 improves to 34.3% vs 23.2% (SASRec) and 29.5% (TIGER), and Live-streaming HR@500 improves to 80.5% vs 68.4% (SASRec) and 73.7% (TIGER) (Table 2).

- **Online A/B impact (business metrics):** the abstract highlights "+14.7% GMV in Product-Card, +10.3% GMV in Short-Video, and +4.9% GMV in Live-Streaming"; later, Table 3 reports detailed lifts (e.g., Product-Card +14.71% GMV; Short-Video +10.33% GMV; Live-Streaming +4.90% GMV).

- **RL effectiveness:** DPO/GRPO improve CTR/CTCVR/GPM over the base model across multiple candidate segments, and GRPO is reported stronger than DPO in their setting (Table 4).

- **Component analyses:** adding FSQ reduces tokenizer conflict (Table 5); Query-Former drastically reduces sequence length and GFLOPs with minor hit-rate changes (Table 6); an auxiliary contrastive objective improves HR metrics by about +1.5%/+1.7%/+1.7% on HR@50/100/500 (Section 4.5).

References (selected; cited/used by the paper)

- Scaling laws / LLM training: OpenAI (GPT-4 technical report), Achiam et al., 2023.

- Generative retrieval in RecSys: TIGER, Rajput et al., 2023.

- Sequential recommendation baseline: SASRec, Kang and McAuley, 2018.

- Vision–language pretraining components: BLIP-2 / Query-Former, Li et al., 2023.

- Contrastive learning objective: InfoNCE / CPC, van den Oord et al., 2018 (as referenced by the paper).

- Finite Scalar Quantization (FSQ): Mentzer et al., 2023.

- Preference optimization / RL: DPO (Rafailov et al., 2023), PPO (Schulman et al., 2017).

- Distributed training techniques: ZeRO (Rajbhandari et al., 2020), Tensor parallelism (Shoeybi et al., 2019).

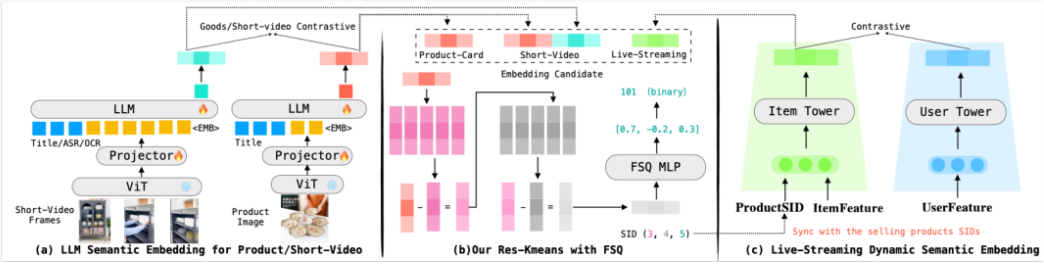

Figure 3: (a) LLM fine-tuning process in Product-Card/Short-Video Item2Item dataset; (c) Live-Streaming embedding generation with selling product Semantic IDs; (b) ResKmeansFSQ Tokenizer, different scenarios utilize different embedding candidates.

3.1.3 Semantic ID Generation Pipeline. Up to now, we have introduced how to train an LLM to generate the semantic embedding $\mathbf{m}$ that with general real world semantic and business knowledge. To transform them as Semantic IDs, we devise an elaborate pipeline for the Product-card, Short-Video and Live-streaming inference, which includes two quantization techniques: Res-Kmeans and FSQ [30]. For the first two layers' Res-Kmeans, we run it as follows:

- *Product-card*: for the product-card, since it only represent commerce intent, thus we only use the LLM generated product-card embedding as $\mathbf{m}$ to quantify.
- *Short-video*: for the e-commerce short-video, it needs to reflect both commerce and watching experience simultaneously, thus we concatenate the LLM generated product-card embedding and the short-video embedding as $\mathbf{m}$ to quantify.
- *Live-streaming*: for live-streaming, its content and selling items dynamically change, thus we could not generate its Semantic ID directly. In OneMall, we utilize the two-tower style retrieval model's item tower generated embedding to quantify, and the real-time selling products' Semantic IDs served as additional feature. Moreover, to alleviate the excessive change rate of semantic codes, we used a lower learning rate for the item tower.

For the last layer FSQ [30], we train a binary 16-bit MLP to quantify the Residual embedding $\mathbf{m} - \mathbf{C}^1_{c_1} - \mathbf{C}^2_{c_2}$ as a 4096 code, it can significantly reduce the conflict rate. For the sake of readability, we do not elaborate on the specific formulas here; an overview of the entire process is illustrated in the accompanying in Figure 3(b)(c).

3.2 Decoder-Only Model Architecture

At OneMall, we employ several Transformer variants served as the backbone architecture, including the Query-Former [22], Cross-Attention, Casual-Self-Attention, as shown in Figure 4. Here we list the designing insights as follows:

- *Query Transformers for Longer Sequence Compression*: In e-commerce service, users always have higher decision-making costs, relying a long funnel of exposure to click to buy an item. As a result, our OneMall needs to utilize a variety of different behavioral sequences to enrich user's preference from different aspects, such as Exposure/Click/Buy sequences and so on. In sequence modeling, if those sequences are concatenated into one, it becomes too long to afford the complexity of Transformer. To alleviate it, we first apply the Query-Former technique to compress the longer sequence into dozens continuous representation:

$$\mathbf{F}_{\texttt{click}} = \texttt{QFormer}_{\texttt{click}}(\mathbf{Q}_{\texttt{click}}, \mathbf{H}_{\texttt{click}}, \mathbf{H}_{\texttt{click}}) \tag{3}$$

where the $\mathbf{F}_{\texttt{click}} \in \mathbb{R}^{M\times D}, \mathbf{Q}_{\texttt{click}} \in \mathbb{R}^{M\times D}, \mathbf{H}_{\texttt{click}} \in \mathbb{R}^{H\times D}$, M denotes the query token number (e.g., 10), H indicates the input sequence length (e.g., 500), D is the embedding dimension (e.g., 128). In this way, our OneMall could consider multiple different sequences with a lower computation cost, such as $\mathbf{F}_{\texttt{Buy}}, \mathbf{F}_{\texttt{Exposure}}$. Besides, we also utilize the Query-Former to compress item side features to generate its representations:

$$\mathbf{f}_{\texttt{item}} = \texttt{MLP}(\texttt{Flatten}(\mathbf{F}_{\texttt{item}})), \quad \mathbf{F}_{\texttt{item}} = \texttt{QFormer}_{\texttt{item}}(\mathbf{Q}_{\texttt{item}}, \mathbf{I}_{\texttt{item}}, \mathbf{I}_{\texttt{item}}) \tag{4}$$

where the $\mathbf{f}_{\texttt{item}} \in \mathbb{R}^D$ is the final item embedding, $\texttt{Flatten}(\cdot)$ is a naive operation to flatten model inputs, $\mathbf{I}_{\texttt{item}}$ is the item-side feature sequence.

- *Cross Transformers for historical information extraction*: Next, to fuse the historical user side information $\{\mathbf{F}_{\texttt{click}}, \mathbf{F}_{\texttt{exposure}}, \mathbf{F}_{\texttt{Buy},\ldots}\}$ to predict next item Semantic IDs $\{s_1, s_2, s_3\}$, we further employ the Cross Attention mechanism for the low latency extraction:

$$\begin{aligned}\{\hat{\mathbf{s}}^L_0, \hat{\mathbf{s}}^L_1, \hat{\mathbf{s}}^L_2, \hat{\mathbf{s}}^L_3\} = \texttt{Cross-Att}^L(\{\mathbf{s}^{L-1}_0, \mathbf{s}^{L-1}_1, \mathbf{s}^{L-1}_2, \mathbf{s}^{L-1}_3\} \cdot \mathbf{W}^L_q, \\ \{\mathbf{F}_{\texttt{click}}, \mathbf{F}_{\texttt{exposure}}, \mathbf{F}_{\texttt{buy},\ldots}\} \cdot \mathbf{W}^L_k, \\ \{\mathbf{F}_{\texttt{click}}, \mathbf{F}_{\texttt{exposure}}, \mathbf{F}_{\texttt{buy},\ldots}\} \cdot \mathbf{W}^L_v)\end{aligned} \tag{5}$$

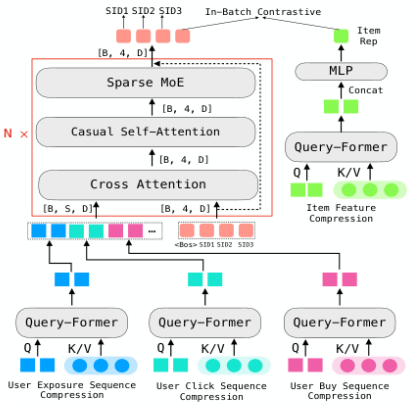

Figure 4: The Transformer backbone of OneMall.

Figure 1: Tokenizer overview from the paper (Figure 3): LLM-based semantic embedding + ResKmeansFSQ semantic IDs across scenarios.

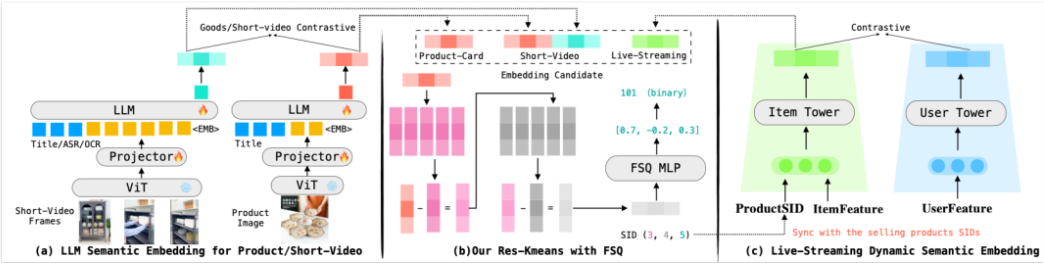

Figure 3: (a) LLM fine-tuning process in Product-Card/Short-Video Item2Item dataset; (c) Live-Streaming embedding generation with selling product Semantic IDs; (b) ResKmeansFSQ Tokenizer, different scenarios utilize different embedding candidates.

3.1.3 Semantic ID Generation Pipeline. Up to now, we have introduced how to train an LLM to generate the semantic embedding $\mathbf{m}$ that with general real world semantic and business knowledge. To transform them as Semantic IDs, we devise an elaborate pipeline for the Product-card, Short-Video and Live-streaming inference, which includes two quantization techniques: Res-Kmeans and FSQ [30]. For the first two layers' Res-Kmeans, we run it as follows:

- *Product-card*: for the product-card, since it only represent commerce intent, thus we only use the LLM generated product-card embedding as $\mathbf{m}$ to quantify.
- *Short-video*: for the e-commerce short-video, it needs to reflect both commerce and watching experience simultaneously, thus we concatenate the LLM generated product-card embedding and the short-video embedding as $\mathbf{m}$ to quantify.
- *Live-streaming*: for live-streaming, its content and selling items dynamically change, thus we could not generate its Semantic ID directly. In OneMall, we utilize the two-tower style retrieval model's item tower generated embedding to quantify, and the real-time selling products' Semantic IDs served as additional feature. Moreover, to alleviate the excessive change rate of semantic codes, we used a lower learning rate for the item tower.

For the last layer FSQ [30], we train a binary 16-bit MLP to quantify the Residual embedding $\mathbf{m} - \mathbf{C}^1_{c_1} - \mathbf{C}^2_{c_2}$ as a 4096 code, it can significantly reduce the conflict rate. For the sake of readability, we do not elaborate on the specific formulas here; an overview of the entire process is illustrated in the accompanying in Figure 3(b)(c).

3.2 Decoder-Only Model Architecture

At OneMall, we employ several Transformer variants served as the backbone architecture, including the Query-Former [22], Cross-Attention, Casual-Self-Attention, as shown in Figure 4. Here we list the designing insights as follows:

- *Query Transformers for Longer Sequence Compression*: In e-commerce service, users always have higher decision-making costs, relying a long funnel of exposure to click to buy an item. As a result, our OneMall needs to utilize a variety of different behavioral sequences to enrich user's preference from different aspects, such as Exposure/Click/Buy sequences and so on. In sequence modeling, if those sequences are concatenated into one, it becomes too long to afford the complexity of Transformer. To alleviate it, we first apply the Query-Former technique to compress the longer

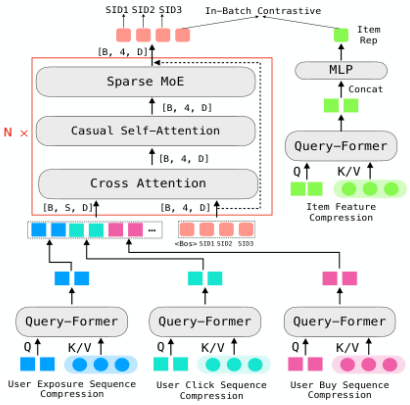

Figure 4: The Transformer backbone of OneMall.

sequence into dozens continuous representation:

$$\mathbf{F}_{\texttt{click}} = \texttt{QFormer}_{\texttt{click}}(\mathbf{Q}_{\texttt{click}}, \mathbf{H}_{\texttt{click}}, \mathbf{H}_{\texttt{click}}) \tag{3}$$

where the $\mathbf{F}_{\texttt{click}} \in \mathbb{R}^{M\times D}, \mathbf{Q}_{\texttt{click}} \in \mathbb{R}^{M\times D}, \mathbf{H}_{\texttt{click}} \in \mathbb{R}^{H\times D}$, M denotes the query token number (e.g., 10), H indicates the input sequence length (e.g., 500), D is the embedding dimension (e.g., 128). In this way, our OneMall could consider multiple different sequences with a lower computation cost, such as $\mathbf{F}_{\texttt{Buy}}, \mathbf{F}_{\texttt{Exposure}}$. Besides, we also utilize the Query-Former to compress item side features to generate its representations:

$$\mathbf{f}_{\texttt{item}} = \texttt{MLP}(\texttt{Flatten}(\mathbf{F}_{\texttt{item}})), \quad \mathbf{F}_{\texttt{item}} = \texttt{QFormer}_{\texttt{item}}(\mathbf{Q}_{\texttt{item}}, \mathbf{I}_{\texttt{item}}, \mathbf{I}_{\texttt{item}}) \tag{4}$$

where the $\mathbf{f}_{\texttt{item}} \in \mathbb{R}^D$ is the final item embedding, $\texttt{Flatten}(\cdot)$ is a naive operation to flatten model inputs, $\mathbf{I}_{\texttt{item}}$ is the item-side feature sequence.

- *Cross Transformers for historical information extraction*: Next, to fuse the historical user side information $\{\mathbf{F}_{\texttt{click}}, \mathbf{F}_{\texttt{exposure}}, \mathbf{F}_{\texttt{Buy},\ldots}\}$ to predict next item Semantic IDs $\{s_1, s_2, s_3\}$, we further employ the Cross Attention mechanism for the low latency extraction:

$$\begin{aligned}\{\hat{\mathbf{s}}^L_0, \hat{\mathbf{s}}^L_1, \hat{\mathbf{s}}^L_2, \hat{\mathbf{s}}^L_3\} = \texttt{Cross-Att}^L(\{\mathbf{s}^{L-1}_0, \mathbf{s}^{L-1}_1, \mathbf{s}^{L-1}_2, \mathbf{s}^{L-1}_3\} \cdot \mathbf{W}^L_q, \\ \{\mathbf{F}_{\texttt{click}}, \mathbf{F}_{\texttt{exposure}}, \mathbf{F}_{\texttt{buy},\ldots}\} \cdot \mathbf{W}^L_k, \\ \{\mathbf{F}_{\texttt{click}}, \mathbf{F}_{\texttt{exposure}}, \mathbf{F}_{\texttt{buy},\ldots}\} \cdot \mathbf{W}^L_v)\end{aligned} \tag{5}$$

Figure 2: Transformer backbone from the paper (Figure 4): Query-Former compression + cross-attention fusion + decoder-only SID generation.